T4 PARTE III

ESTRUTURA DE DADOS:

- Em *geração.c*: a *struct geração* utiliza um vetor de indivíduos e de espécies com tamanho fixo. Poderiam ser substituídos por **vetores alocados dinamicamente**. Também pensei em usar listas encadeadas, no entanto o acesso direto a elementos aleatórios usado em algumas funções seria custoso, e a ordenação também.
- Em *espécie.c*: A *struct espécie* possui um vetor de indivíduos de tamanho fixo. Novamente poderia ser substituído por um **vetor alocado dinamicamente**.
- Em *indivíduo.c:* A parte de inovações poderia ser transformada em algum tipo de árvore binária de busca, isso não só tiraria a restrição de *MAX_INOVAÇÕES* mas também tornaria as buscas em *id_neurônio_entre* mais rápida. A comparação dessa árvore poderia ser feita comparando primeiro os id's de origem, se o buscado for menor vai pra esquerda, se for maior vai pra direita e se forem iguais compara os id's de destino da mesma forma. E a preocupação com o caso dos pares de id's serem iguais não é complicada para a implementação, por que se os 2 forem iguais justamente é só retornar o id_meio e não criar nada. Poderia implementar uma Treap (estudei em estrutura de dados) para evitar degeneração (ABB ficar desbalanceada), mas pelo meu entendimento o risco de uma ABB convencional degenerar não é tão real nesse código.

API:

- Em *grafo.c* poderia existir uma função para alterar, uma para criar e remover aresta. Atualmente elas são a mesma função (mesma função fazendo três coisas), para o programa em questão não há problema mas é menos modularizado, ou pelo menos ter um retorno ou parâmetro que indicasse caso a aresta foi criada, removida ou somente alterado o valor, pois isso pode ser útil.
- Em *rede.c, rede_insere_sinapse* cria uma aresta, verifica se gerou ciclo, se gerou remove a aresta criada e se não gerou apenas retorna. Poderia existir uma função em *grafo.c* que verifica diretamente se existe um caminho entre um nó e outro (dai bastaria verificar se já existe caminho entre o destino e a origem, por que isso implica que inserir uma aresta de origem pra destino gera ciclo), assim não precisaria criar e remover desnecessariamente uma aresta e, em alguns casos, tornaria essa função de inserir sinapse mais rápida, pois já vai direto aonde pode ser criado o problema.